

Exercício 1

Main.java RUN

```
1  class Animal {
2      String som() { return "..."; }
3  }
4  class Cachorro extends Animal {
5      String som() { return "au"; }
6  }
7  public class Main {
8      public static void main(String[] args) {
9          Animal x = new Cachorro();
10         System.out.println(x.som());
11     }
12 }
```

OUTPUT

au

(a) Saída: au. (b) Decisão: na execução. Em Java os métodos de instância são dinâmicos por padrão: o que manda é o tipo do *objeto* (Cachorro), não o da variável (Animal). Aqui o Animal é só a “etiqueta” que a variável usa.

Exercício 2

main.cpp RUN

```
1  #include <iostream>
2  using namespace std;
3
4  struct A {
5      void f() { cout << "A"; }
6  };
7  struct B : A {
8      void f() { cout << "B"; }
9  };
10
11 int main() {
12     B b;
13     A *p = &b;
14     p->f();
15     return 0;
16 }
```

OUTPUT

A

(a) Saída: A. (b) Decisão: na compilação. Em C++ o padrão é a vinculação estática: como `f` não é virtual, o compilador olha o tipo do ponteiro (`A*`) e já amarra a chamada ao `f` de A. O fato de o objeto ser um B não entra na conta.

main.cpp RUN

```
1  #include <iostream>
2  using namespace std;
3
4  struct A {
5      virtual void f() { cout << "A"; }
6  };
7  struct B : A {
8      void f() { cout << "B"; }
9  };
10
11 int main() {
12     B b;
13     A *p = &b;
14     p->f();
15     return 0;
16 }
```

OUTPUT

B

Agora a saída é B: com `virtual`, a escolha passa a ser feita na execução, consultando a vtable do objeto (o assunto do slide 20).

Exercício 3

Main.java RUN

```
1  class A {
2      String nome = "A";
3      String getNome() { return nome; }
4  }
5  class B extends A {
6      String nome = "B";
7      String getNome() { return nome; }
8  }
9  public class Main {
10     public static void main(String[] args) {
11         A x = new B();
12         System.out.println(x.nome + " " + x.getNome());
13     }
14 }
```

OUTPUT

A B

(a) Saída: A B. **(b) Decisão:** o campo foi decidido na **compilação**, e o método na **execução**.

O `x.nome` usa o tipo declarado da variável (A), então devolve o campo de A. O campo `nome` de B não substitui o de A, ele apenas o *esconde*, e são duas variáveis diferentes dentro do mesmo objeto. Já `x.getNome()` é um método de instância, despachado dinamicamente: roda a versão de B, que devolve o `nome` de B. É um bom motivo para manter os campos privados e só expor métodos.

Exercício 4

main.py RUN

```
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador(); b = Contador()
8 print(a.id, b.id, a.total)
```

OUTPUT

1 2 2

(a) Saída: 1 2 2. (b) Decisão: na execução, já que em Python tudo é resolvido na hora, procurando o nome no objeto e depois na classe.

O `total` é uma variável de **classe**: existe uma só cópia, compartilhada por todos. O `id` é de **instância**: cada objeto tem o seu. Por isso `a.id` vale 1, `b.id` vale 2, e `a.total` não encontra nada com esse nome no objeto, sobe para a classe e acha o 2 (o mesmo valor que `b.total` mostraria).

Exercício 5

Main.java RUN

```
1 class A {
2     static String quem() { return "A"; }
3 }
4 class B extends A {
5     static String quem() { return "B"; }
6 }
7 public class Main {
8     public static void main(String[] args) {
9         A x = new B();
10        System.out.println(x.quem());
11    }
12 }
```

OUTPUT

A

(a) Saída: A. (b) Decisão: na compilação. Métodos `static` pertencem à classe, não ao objeto, então não são sobrescritos, e sim *escondidos*. A chamada é resolvida pelo tipo declarado (A). O compilador ou a IDE costuma avisar para escrever `A.quem()`, que deixa claro o que acontece.

Exercício 6

main.go RUN

```
1 package main
2
3 import "fmt"
4
5 type Animal struct{}
6
7 func (Animal) Som() string { return "... " }
8 func (a Animal) Falar() string {
9     return "faz " + a.Som()
10 }
11
12 type Cao struct{ Animal }
13
14 func (Cao) Som() string { return "au" }
15
16 func main() {
17     fmt.Println(Cao{}.Falar(), Cao{}.Som())
18 }
```

OUTPUT

faz ... au

(a) Saída: faz ... au. **(b) Decisão:** na **compilação**.

Dentro de `Animal.Falar`, o `a` é um `Animal` de verdade, e não um `Cao` disfarçado, então `a.Som()` chama o `Som` de `Animal` e devolve "... ". Entre o struct embutido e o que o contém não há despacho dinâmico: o embedding só “promove” os métodos, o que é diferente de herança. Já `Cao{}.Som()` chama direto o `Som` de `Cao`. Para ter polimorfismo de verdade em Go, é preciso usar uma **interface**